KI-gestütztes HR-Recruiting & Skill-Matching ("TalentMatch")

Eine Anwendung, die den Bewerbungsprozess optimiert, indem sie eingehende Lebensläufe analysiert, mit Stellenprofilen abgleicht und automatisierte Interviewfragen generiert.

- **Backend (Spring Boot):**
    - Strukturierte JSON-Ausgabe des LLMs (Structured Outputs) zur automatisierten Extraktion von Skills, Berufserfahrung und Ausbildung aus PDFs.
    - Ein Matching-Algorithmus, der das Profil des Bewerbers mit den Anforderungen der Vakanz vergleicht.
    - REST-API mit Spring Security (JWT) und Rollenverwaltung (HR-Manager vs. Bewerber).
- **Frontend (Angular):**
    - Drag-and-Drop-Upload für Lebensläufe.
    - Übersichtlich gestaltetes Kanban-Board für den Bewerbungsstatus.
    - Detaillierte Ansicht mit einem KI-generierten Matching-Score, Stärken-Schwächen-Analyse und vorgeschlagenen Interviewfragen für den HR-Manager.
- **Warum es überzeugt:** Es löst ein reales Geschäftsproblem, demonstriert den Umgang mit strukturierten KI-Daten (Function Calling / JSON Mode) und bietet eine anspruchsvolle Angular-UI (Kanban, komplexe Dashboards).

Tipps für die Umsetzung im Portfolio

- **Spring AI nutzen:** Verzichte im Backend möglichst auf rohe HTTP-Client-Aufrufe an OpenAI und nutze stattdessen das offizielle **Spring AI**-Projekt. Das zeigt, dass du dich im Java-Ökosystem auf dem neuesten Stand befindest.
- **Clean Code & Architektur:** Achte auf eine saubere Schichtarchitektur (Controller, Service, Repository) im Backend und modular aufgebaute, reaktive Komponenten im Angular-Frontend.
- **Deployment:** Packe die Anwendung in Docker-Container und stelle sie bereit (z. B. via Docker Compose auf einem kleinen Cloud-Server oder kostenlos auf Plattformen wie Render/Railway), damit Recruiter direkt einen Live-Link ausprobieren können.

Das ist eine hervorragende Wahl! **"TalentMatch"** ist ein starkes Showcase-Projekt, weil es echte Business-Logik mit modernen KI-Konzepten wie **Structured Outputs**, **PDF-Parsing** und **Vector-based / Rule-based Matching** verbindet.

Systemkonzept: TalentMatch – AI Recruiting Assistant

1. Zielsetzung & Zielgruppe

- **Ziel:** Die Automatisierung und Unterstützung des Sichtungsprozesses von Bewerbungen für HR-Teams.

- **Kernfunktionen:**

- Automatisches Extrahieren struktureller Daten aus Resume-PDFs.

- Bewertung der Eignung bezüglich einer spezifischen Stellenbeschreibung (Matching Score & Begründung).

- Generierung von maßgeschneiderten Interviewfragen basierend auf Wissenslücken/Unklarheiten im Lebenslauf.

2. Systemarchitektur & Tech-Stack

Frontend: Angular (v17+)

- **State Management & Data Flow:** Angular Signals, RxJS für asynchrone Events.
- **Architecture:** Standalone Components, Reactive Forms, Route Guards.
- **UI Component Library:** Angular Material oder Tailwind CSS (für modernes Kanban/Dashboard Layout).
- **Key Features:** File Drag-and-Drop, Kanban-Board für Bewerbungsphasen, interaktive Score-Visualisierung.

Backend: Java 17+ / Spring Boot 3+

Frameworks & Libs:

- **Spring AI:** Anbindung an LLMs (OpenAI gpt-4o oder Ollama llama3), Prompt-Templates und Structured Output Converters.
- **Spring Security + JWT:** Authentifizierung für HR-Manager.
- **Spring Data JPA & PostgreSQL:** Relationale Speicherung von Stellenangeboten, Bewerbern und Matches.
- **Apache Tika / PDFBox:** Zuverlässiges Extrahieren von Text aus PDF-Lebensläufen.

3. Datenmodell & Entitäten (Domain Model)

1. **JobPosition (Stellenanzeige)**

- id, title, department, description, requiredSkills (List), experienceLevel, createdDate

2. **Candidate (Bewerber/in)**

- id, firstName, lastName, email, phone, rawResumeText, parsedProfile (JSON)

3. **Application (Bewerbung)**

- id, jobPositionId, candidateId, status (NEW, MATCHED, INTERVIEW, REJECTED, HIRED), createdAt

4. **AnalysisResult (KI-Evaluierung)**

- id, applicationId, overallScore (0-100), summary, matchingSkills (List), missingSkills (List), suggestedQuestions (List), riskFactors (List)

4. KI-Workflows mit Spring AI

Workflow 1: Resume Parsing (Structured Output)

- **Input:** PDF-Datei.
- **Prozess:**

1. PDF-Text wird mit Apache Tika / PDFBox extrahiert.
2. Spring AI schickt den Text an das LLM mit einem `BeanOutputConverter<CandidateProfileDto>`.

- **Output:** Ein valides Java-Objekt mit extrahierten Feldern:

1. Name, E-Mail, bisherige Positionen, Technologien, Ausbildungsabschluss, Sprachen.

Workflow 2: Match Analysis & Scoring

- **Input:** Extrahierter Lebenslauf + JobPosition-Anforderungen.
- **Prozess:**

1. System promptet das LLM: *"Vergleiche Bewerber-Profil A mit Anforderungsprofil B."*
2. Spring AI zwingt das Modell zur Rückgabe von `AnalysisResultDto`.

- **Output:**

1. overallScore: Z.B. 85/100
2. matchingSkills: ["Java", "Spring Boot", "Angular"]
3. missingSkills: ["Kubernetes", "AWS"]
4. suggestedQuestions: ["Wie tiefgehend sind die Kenntnisse im Bereich Mikrodienste?", "Erfahrung mit Angular Signals vorhanden?"]

5. UI/UX Design Blueprint (Für das Frontend-Design)

1. Dashboard / Stellenübersicht

- Übersicht aller offenen Vakanzen mit Anzahl der eingegangenen Bewerbungen.
- Kachel- oder Tabellenansicht.

2. Kanban-Board (Bewerbungs-Pipeline)

- Spalten: Eingegangen (Neu) -> KI-Analysiert -> Interview -> Zusage / Absage.
- Bewerber-Karten zeigen Name, Match-Score (Farbig: Grün/Gelb/Rot) und Haupt-Skills.

3. Bewerber-Detailansicht (AI Analytics View)

- **Header:** Bewerbername, Match-Score Badge, Kontaktdaten.
- **Left Column:** PDF-Vorschau oder extrahierter Werdegang.
- **Right Column (AI Insights):**
- **Pro / Contra Matrix:** Gefundene vs. fehlende Skills.
- **KI-Zusammenfassung:** 3–4 Sätze Management Summary.
- **Interview-Helper:** Generierte Fragen inklusive Checkboxen für das Gespräch.

6. Schritt-für-Schritt Roadmap zur Umsetzung

1. **Phase 1: Backend Setup & Core Domain**

- Spring Boot Projekt mit Spring AI, JPA und PostgreSQL initialisieren.
- Entities und REST-Endpoints für Jobs & Applications anlegen.

2. **Phase 2: Spring AI Pipeline**

- PDF-Parsing-Service bauen.
- Prompts und Structured Output Converters für Parsing & Matching implementieren.

3. **Phase 3: Angular Frontend**

- Angular App strukturieren (Services, Components, Signal Stores).
- Kanban Board mit Drag-and-Drop (@angular/cdk/drag-drop) bauen.
- AI-Insight-Komponenten visualisieren.

4. **Phase 4: Docker & Demo-Vorarbeitung**

- docker-compose.yml für App, Datenbank & ggf. lokalen Ollama-Server.
- Test-Daten (Szenarios: Top-Candidate, Mid-Candidate, Mismatch) vorbereiten.